
SUNRISE AI CONSULTANT

Technical Architecture & System Design Document

Author: AI Coding Assistant

Date: July 11, 2026

Target System: Sunrise AI Consultant Platform

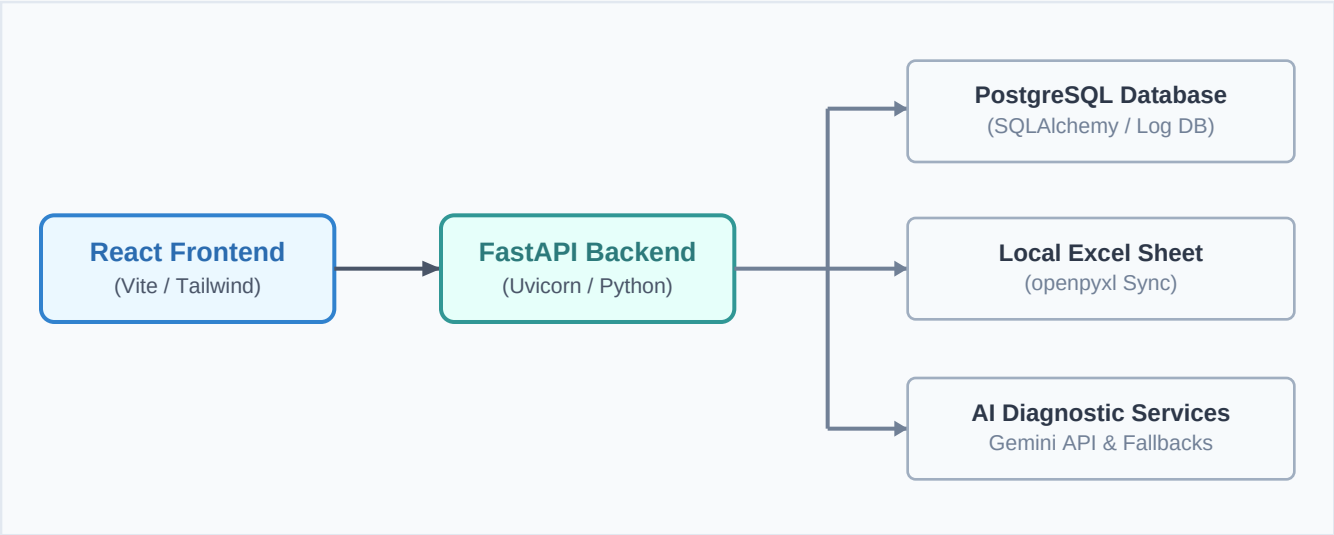
Status: Approved / Implemented

1. Executive Summary

The **Sunrise AI Consultant** is an intelligent, full-stack questionnaire application designed to recommend tailored software plans and add-on modules from the Sunrise business suite. The system targets tailoring workshops, retail shops, cattle feed mills, and general small-to-medium businesses. By capturing specific metrics (such as order volumes, employee counts, and operation bottlenecks), the platform leverages Google's Gemini GenAI service to deliver highly contextualized business diagnostics. The system ensures robust operations through database logging and offline local spreadsheet synchronization.

2. System Architecture Diagram

The block diagram below maps the relationships and data flows between components:



3. Core Components

3.1 Frontend Web Interface (React & Vite)

The frontend is a single-page application built on **React 18** and bundled via **Vite**. It features an interactive chat interface that guides users through a tailored set of questions loaded from domain-specific configuration files (tailoring, retail, cattlefeed, or general). Key highlights include:

- **Dynamic Questioning:** Questions adapt based on selected business category and keywords.
- **State Tracking:** Maintains user responses locally until final questionnaire completion.
- **Report Dashboard:** Renders interactive analytics, tier pricing rationales, and customized recommendations.

3.2 Backend Server (FastAPI)

The backend server is powered by **FastAPI** and served with **Uvicorn**. It implements asynchronous handlers and exposes clean REST endpoints. Key integrations are described below:

Integration Layer	Technology	Description / Fallback Strategy
Primary GenAI	Google Gemini API	Generates structured JSON business suggestions based on answers and templates
Secondary Fallback	OpenRouter (GPT-4o)	Engages automatically if the Google Gemini API returns an error or rate limit.
Offline Fallback	Python Rule Engine	Executes custom local business heuristics if both GenAI endpoints are unreachable
Database Storage	PostgreSQL (SQLAlchemy)	Stores full questionnaire payload (JSON answers) and suggestions (JSON results)
Spreadsheet Sync	Excel (openpyxl)	Syncs answers and recommendations to separate worksheets in a shared local workbook

4. End-to-End Data Flow

- When a user completes their business questionnaire, the system follows a structured transaction sequence:
- Frontend Submission:** The client bundles questionnaire answers and posts them to `/api/submit-answers`.
 - AI Recommendation Phase:** Backend invokes `gemini_service.py` which queries the Gemini API with structured schemas. If unsuccessful, it cascades to OpenRouter or offline heuristics.
 - SQL Registry Phase:** The diagnostic data, including JSON questions/answers and suggestions, is saved to the PostgreSQL `business_diagnostics` table via SQLAlchemy.
 - Local Excel Backup Phase:** The backend asynchronously runs a local sheet sync. The file `consultant_data.xlsx` is loaded or created, and the data is formatted and saved using openpyxl.
 - Unified Response Dispatch:** The API returns a unified JSON payload detailing the recommendations, database registry ID, and the Excel backup operation status.

5. Technical Stack Summary

Component	Technologies & Frameworks	Key Purpose
Frontend	React 18, Vite, ES6+, Tailwind CSS	User Interaction & Dashboard Rendering
Backend	Python 3.11, FastAPI, Uvicorn	REST API & Core Logic Routing
Database	PostgreSQL, SQLAlchemy, Alembic	Structured relational log storage
File Generation	openpyxl, ReportLab	Excel Syncing & System PDF Generation
AI Orchestration	requests, google-auth, json-schema	Integrations with Gemini & OpenRouter APIs